

Guía de *Skills* para Creadores de Contenido

Dando instrucciones a OpenCode para que siga tus reglas

JA Palazón Ferrando

2026-06-21

Contenidos

1	¿Qué es un <i>skill</i> en OpenCode?	2
2	Agente vs <i>Skill</i>	2
3	Cuándo usar <i>skill</i> vs agente	2
4	Anatomía de un <i>skill</i>	3
5	Cómo se activa un <i>skill</i>	4
6	Ejemplo práctico	4
7	Ubicación de los <i>skills</i>	4
8	Consejos para escribir <i>descriptions</i> efectivas	5
8.1	Haz esto	5
8.2	Tabla de patrones recomendados	5
9	Cómo empezar en 3 pasos	5
9.1	Paso 1: Crea el directorio	5
9.2	Paso 2: Crea el archivo	6
9.3	Paso 3: Escribe y guarda	6
9.4	Pruébalo	6
10	Errores comunes	6
11	Consejos rápidos	6
12	Resumen rápido	6
13	Referencias	7
A	Anexo: Skill <i>quarto-authoring</i> para copiar	7
A.1	Descarga el fichero	7
A.2	Qué incluye este skill	7
A.3	Ejemplo: <i>callouts</i>	7
A.4	Otros elementos	8
A.5	Sugerencias de personalización	8
A.5.1	Cambiar la <i>description</i>	8
A.5.2	Añadir reglas específicas	8

💡 ¿Para quién es esta guía?

Esta guía amplía la información sobre *skills* de OpenCode para creadores de contenido que ya tienen algo de experiencia. **No necesitas saber programar** para entenderla.

1. ¿Qué es un *skill* en OpenCode?

Un *skill* (habilidad o destreza) es un conjunto de instrucciones escritas que OpenCode carga automáticamente cuando detecta que la tarea que le pides encaja con el propósito del *skill*.

Piénsalo como una *guía de referencia o manual de instrucciones*:

- Le dice al modelo *cómo* abordar cierto tipo de tareas.
- Le recuerda *qué* convenciones, reglas o pasos seguir.
- Se activa solo, sin que tengas que mencionarlo.

i Nota

Los *skills* no son código ejecutable. Son **texto instructivo** que el modelo lee antes de responder. Es como darle a un asistente un manual de procedimientos antes de que empiece a trabajar.

2. Agente vs *Skill*

Aspecto	Agente	<i>Skill</i>
¿Qué es?	Un asistente con rol y personalidad definidos	Instrucciones + flujo de trabajo
¿Cómo se activa?	Manualmente, escribiendo @nombre	Automáticamente, según la tarea
¿Tiene herramientas propias?	Sí, puede tener acceso a herramientas y permisos	No, solo aporta contexto textual
¿Puede ejecutar acciones?	Sí, actúa por sí mismo	No, solo guía al modelo principal
Ejemplo	@editor que revisa ortografía	<i>Skill</i> que aplica guía de estilo automáticamente

⚠ Aviso

Los *skills* **no se invocan manualmente**. Si necesitas activar algo explícitamente, probablemente necesitas un **agente**, no un *skill*.

3. Cuándo usar *skill* vs agente

Usa un **skill** cuando:

- Tienes conocimiento contextual que debe aplicarse cada vez que aparece cierto tipo de tarea (ej.: “cada vez que generes documentación, sigue estas reglas de estilo”).
- Quieres que el modelo recuerde convenciones sin que tú las tengas que escribir cada vez.
- El flujo de trabajo es predecible y repetitivo.

Usa un **agente** cuando:

- Necesitas que alguien (aunque sea IA) tome un rol activo: revisar, editar, proponer cambios.
- Quieres invocar una capacidad concreta escribiendo @nombre.

- El agente necesita herramientas específicas (navegador, shell, etc.).

💡 Consejo

Regla práctica:

skill = *qué saber* (conocimiento pasivo)

agente = *qué hacer* (acción activa)

4. Anatomía de un *skill*

Cada *skill* vive dentro de un directorio con su nombre:

```
.opencode/skills/<nombre>/SKILL.md
```

Ejemplo de estructura:

```
.opencode/skills/  
  revisar-estilo/  
    SKILL.md
```

El archivo *SKILL.md* tiene dos partes:

1. **Frontmatter YAML** — metadatos que OpenCode lee para saber cuándo activar el *skill*.
2. **Cuerpo en Markdown** — las instrucciones, ejemplos y flujos de trabajo.

```
---  
name: revisar-estilo  
description: >  
  Revisa que la documentación generada siga la guía de estilo interna.  
  Usa cuando se genere documentación nueva o se modifique la existente.  
---  
  
# Skill: Revisión de Estilo  
  
## Reglas generales  
  
- Tono: claro, directo, sin jerga innecesaria.  
- Todas las cabeceras usan formato "Title Case".  
- Los ejemplos de código deben incluir el lenguaje en el bloque.  
  
## Flujo de trabajo  
  
1. Lee el documento generado.  
2. Identifica infracciones de estilo.  
3. Sugiere correcciones específicas.  
4. No modifiques el contenido sin permiso explícito.
```

i Nota

El **frontmatter** es el bloque de metadatos entre `---` al inicio del archivo. OpenCode lo usa para decidir si el *skill* es relevante para la tarea actual. El **cuerpo** es lo que se inyecta en el contexto del modelo.

5. Cómo se activa un *skill*

El modelo principal de OpenCode decide automáticamente si cargar un *skill*. Lo hace comparando la *description* del *skill* con la tarea que le acabas de pedir.

No puedes forzar la activación de un *skill*. Depende de:

- Que la *description* del *skill* coincida con la tarea.
- Que el modelo considere relevante el *skill* en ese momento.

Aviso

Si necesitas asegurarte de que algo se cargue siempre, considera usar **instrucciones del proyecto** en lugar de un *skill*. Los *skills* son contextuales, no obligatorios.

6. Ejemplo práctico

Skill para **documentalistas** que quieren mantener una guía de estilo coherente.

Archivo: `.opencode/skills/estilo-documentacion/SKILL.md`

```
---
name: estilo-documentacion
description: >
  Aplica la guía de estilo de documentación técnica.
  Usa cuando generes, edites o revises documentación en español.
  Usa SOLO cuando trabajes con archivos .qmd, .md o .Rmd.
---

# Guía de Estilo para Documentación Técnica

## Voz y tono

- Usa voz activa: "El sistema procesa los datos" (no "los datos son procesados").
- Trata a la persona lectora de "tú".
- Sé concisa: una idea por párrafo.

## Convenciones de formato

- **Títulos**: `## Title Case` (nunca todo mayúsculas).
- **Código**: usa bloques con nombre de lenguaje: ``python`.
- **Énfasis**: usa *cursiva* para términos nuevos, **negrita** para acciones importantes.

## Lo que NO debes hacer

- No uses notas al pie en documentación técnica.
- No uses emojis a menos que la usuaria lo solicite.
- No añadas jerga innecesaria.

## Flujo de revisión

1. Identifica el tipo de archivo (`.qmd`, `.md`, `.Rmd`).
2. Revisa tono, formato y convenciones según esta guía.
3. Si encuentras una infracción, señala la línea y sugiere una corrección.
4. Pregunta antes de aplicar cambios automáticamente.
```

7. Ubicación de los *skills*

Hay dos lugares donde puedes colocar *skills*:

Ubicación	Alcance	Ejemplo de ruta
Proyecto	Solo afecta a ese proyecto	<code>.opencode/skills/<nombre>/SKILL.md</code>
Global	Afecta a todos los proyectos	<code>~/.config/opencode/skills/<nombre>/SKILL.md</code>

- Los *skills* de **proyecto** van dentro del repositorio o directorio del proyecto. Son ideales para convenciones de equipo.
- Los *skills* **globales** van en tu directorio personal de configuración. Sirven para reglas que aplicas siempre, como tu guía de estilo personal.

Consejo

Si trabajas con un equipo, usa *skills* de proyecto y versiona el directorio `.opencode/` con Git. Así todo el mundo comparte las mismas reglas.

8. Consejos para escribir *descriptions* efectivas

La *description* es lo que OpenCode usa para decidir si cargar tu *skill*. Una buena *description* marca la diferencia.

8.1. Haz esto

- **Pon las palabras clave al inicio.**
 "Revisa que la documentación siga la guía de estilo..."
 "Skill para cuando necesites que alguien revise..."
- **Sé específica.**
 "Usa cuando generes archivos `.qmd` o `.md` en español."
 "Para documentación."
- **Usa frases como "Usa cuando ..."** o **"Usa SOLO cuando ..."**.
 Esto ayuda al modelo a emparejar el skill con la tarea.
- **Menciona el tipo de archivo, lenguaje o formato.**
 Ejemplo: "Usa SOLO cuando trabajes con archivos Python (`.py`)."

8.2. Tabla de patrones recomendados

Patrón	Ejemplo
Usa cuando...	Usa cuando revises documentación existente.
Usa SOLO cuando ...	Usa SOLO cuando modifiques archivos <code>.qmd</code> ...
Usa para...	Usa para aplicar convenciones de escritura inclusiva.

Aviso

Una descripción demasiado genérica hará que el skill se active en contextos inapropiados o que no se active nunca. Invertir tiempo en una buena descripción es clave.

9. Cómo empezar en 3 pasos

9.1. Paso 1: Crea el directorio

Los *skills* se guardan en `.opencode/skills/<nombre>/` dentro de tu proyecto. Si no existe, créala.

9.2. Paso 2: Crea el archivo

Cada *skill* es un archivo *SKILL.md* dentro de su directorio:

```
.opencode/skills/estilo-documentacion/SKILL.md
.opencode/skills/revisar-textos/SKILL.md
```

9.3. Paso 3: Escribe y guarda

Escribe el *frontmatter* YAML (con `---`) y las instrucciones en el cuerpo. Guarda el archivo.

9.4. Pruébalo

Abre OpenCode y realiza una tarea que coincida con la *description* de tu *skill*. Si la *description* es buena, el *skill* se cargará automáticamente.

i Archivos ocultos

El directorio `.opencode` comienza con un punto. En muchos sistemas está oculto. Si no lo ves, activa “mostrar archivos ocultos” en tu explorador.

10. Errores comunes

Error	Consecuencia	Solución
<i>Description</i> demasiado genérica	El <i>skill</i> se activa en contextos inapropiados	Sé específico y usa “Usa cuando ...”
No guardar en <code>.opencode/skills/<nombre>/</code>	OpenCode no encuentra el <i>skill</i>	Verifica la ruta exacta
Instrucciones vagas en el cuerpo	El modelo no sabe qué hacer	Sé claro y usa ejemplos
Mezclar varios temas en un <i>skill</i>	Resultados inconsistentes	Crea un <i>skill</i> por tema
Usar <i>skill</i> cuando necesitas agente	No se activa o no ejecuta acciones	Usa <code>@nombre</code> para invocar un agente

11. Consejos rápidos

- **Un *skill*, un tema.** No metas reglas de estilo, traducción y revisión en el mismo *skill*.
- **Empieza simple.** Un *skill* con 5 reglas claras es mejor que uno con 50 confusas.
- **Prueba y ajusta.** Los *skills* no salen perfectos a la primera. Observa cuándo se activan y ajusta la *description*.
- **Copia y adapta.** Si un *skill* te funciona bien, cópialo y cambia solo lo necesario para otro contexto.

12. Resumen rápido

Concepto	Clave
<i>Skill</i>	Instrucciones que se cargan automáticamente según la tarea
Activación	Automática, la decide el modelo
Estructura	<i>Frontmatter</i> YAML (metadatos) + cuerpo en <i>Markdown</i> (instrucciones)

Concepto	Clave
Ubicación	Proyecto (<code>.opencode/</code>) o global (<code>~/.config/opencode/</code>)
<i>Description</i>	Debe ser específica, con <i>keywords</i> al inicio
Diferencia con agente	<i>Skill</i> = conocimiento pasivo; agente = rol activo con herramientas

13. Referencias

- [Documentación de Skills](#) — Cómo crear y usar skills
- [Documentación oficial de OpenCode](#)

A. Anexo: Skill *quarto-authoring* para copiar

A.1. Descarga el fichero

El fichero completo del skill está en: [quarto-authoring-SKILL.md](#)

1. Ábrelo con tu editor de texto
2. Estudia el contenido
3. Si te gusta (o lo modificas), cópialo a `.opencode/skills/quarto-authoring/SKILL.md`

A.2. Qué incluye este skill

Este skill le indica a OpenCode cómo trabajar con documentos *Quarto*. Cubre:

- *Callouts* (bloques de nota, advertencia, consejo)
- *Cross-references* (referencias a figuras, tablas, secciones, ecuaciones)
- Tablas y figuras con *captions* y *alt text*
- *Citations* (referencias bibliográficas)
- *Divs* y *spans* (contenedor con formato)
- Configuración de documentos HTML, PDF y presentaciones *RevealJS*

A.3. Ejemplo: *callouts*

Los *callouts* son bloques destacados para notas, advertencias o consejos:

```

::: {.callout-note}
Esta es una nota informativa.
:::

::: {.callout-warning}
## Título personalizado
Esta es una advertencia.
:::

```

Tipos disponibles: `note`, `warning`, `important`, `tip`, `caution`.

A.4. Otros elementos

- **Cross-references:** usa @fig-nombre, @tbl-nombre, @sec-nombre, @eq-nombre para referenciar elementos.
- **Tablas:** define una tabla con *caption* usando `::: {#tbl-ejemplo}`.
- **Figuras:** incluye `fig-alt` para accesibilidad: `![Caption](image.png){#fig-name fig-alt="Alt text"}`.
- **Citations:** referencia bibliográfica con @autor2020 o [@autor2020; @autor2021].

Consulta el fichero completo para ver todos los detalles y *workflows* comunes.

A.5. Sugerencias de personalización

A.5.1. Cambiar la *description*

Adapta la descripción a tu tipo de documentos:

Tipo de documento	Description personalizada
Informes técnicos	“Usa cuando generes informes técnicos en formato <i>.qmd</i> ”
Tutoriales	“Usa cuando crees tutoriales o guías paso a paso”
Artículos académicos	“Usa cuando trabajes con artículos que incluyan <i>citations</i> y referencias”

A.5.2. Añadir reglas específicas

Puedes añadir reglas de estilo en el cuerpo del skill:

```
# Guía de Estilo para Quarto

## Convenciones de formato

- Usa voz activa en los textos.
- Trata a la persona lectora de "tú".
- Incluye `fig-alt` en todas las figuras.
- Usa `#| label:` en todos los bloques de código.
```

A.6. Preguntas de reflexión

Antes de crear tu propio skill, considera estas preguntas:

1. **¿Qué tipo de documentos creas con más frecuencia?** Artículos, tutoriales, informes ... El tipo define qué skill necesitas.
2. **¿Qué elementos de *Quarto* usas siempre?** *Callouts*, *cross-references*, tablas ... El skill puede recordarte las convenciones.
3. **¿Qué errores quieres evitar?** Revisa la tabla de errores comunes y piensa cómo los aplicarías a tu caso.
4. **¿Cuándo debería activarse el skill?** Piensa en frases clave para la *description*: “Usa cuando generes archivos *.qmd*”.

i Consejo

Respóndelas antes de copiar el ejemplo del anexo. Te ayudarán a personalizar tu skill desde el principio.

Si lo prefieres, usa esta configuración inicial y ve experimentando versiones. Así podrás comprobar el efecto de tus cambios.

*Documento creado por el agente **escribidor** — un asistente de documentación técnica para OpenCode.*

Curso completo disponible en: palazon.github.io/ocCursoAvanzado